Sesiunea 6: Workshop Practic - Finalizarea Aplicației "Task Tracker"

Capitolul 1: Construirea Scheletului (HTML5) - Fundația pentru Automatizare 2

- 🔴 Pasul 1: Boilerplate-ul și Header-ul (Navigația) 2
- 🔴 Pasul 2: Secțiunea de Input (Formularul) și Atributele de Testare 3
- 🔴 Pasul 3: Pregătirea Listei și Indicatorul de Latență (Spinner) 4

Capitolul 2: Prezentarea Vizuală (CSS3) - Construirea interfeței pas cu pas 5

- 🔴 Pasul 1: Resetarea și Flexbox-ul pe Navbar 6
- 🔴 Pasul 2: Formularul și Stările Butonului (:disabled vs :hover) 7
- 🔴 Pasul 3: Pregătirea Lista de Task-uri (Latența & Lista) 8

Capitolul 3: Logica Aplicației (JavaScript) - "Creierul" Asincron 10

- 🔴 Pasul 1: Selectoarele DOM și Funcția de Randare (Reutilizabilitate) 10
- 🔴 Pasul 2: Încărcarea datelor la pornire (GET) 11
- 🔴 Pasul 3: Salvarea unui Task Nou (POST & Latență) 12
- 🔴 Pasul 4: Validarea Formularului (Evenimentul de 'input') 14

Capitolul 4: Ștergerea pe Rețea (DELETE) și Checklist-ul Final de QA 16

- 🔴 Pasul 1: Ștergerea reală (Metoda HTTP DELETE & Event Delegation) 16
- 🔴 Pasul 2: Manual QA Checklist (SUT Sign-off) 18

Capitolul 5 (Bonus): Modul "Chaos" - Inamicul testelor automate (Flaky Pop-up) 19

- 🔴 Pasul 1: Stilizarea Pop-up-ului (Z-Index și Overlays) 20
- 🔴 Pasul 2: JavaScript Asincron Aleatoriu (Injecția) 21
- 🔴 Concluzia QA-ului (SUT Sign-off) 23

Workbook S6 - Extinderea Aplicației "Task Tracker" 23

- 💻 Exercițiul 1: Contorul de Task-uri (State Management) 24
- 💻 Exercițiul 2: Bara de Căutare (Filtrare live în DOM) 24
- 💻 Exercițiul 3: "Șterge Tot" prin Paralelizare (Senior Level) 25

💻 Exercițiul 4: Finalizarea unui Task (Metoda PATCH) 25
💻 Exercițiul 5: Sortare Alfabetică în DOM 26
💻 Exercițiul 6: Sistem de "Retry" (Auto-Vindecare) 26
🔑 Cheile de Rezolvare (Soluții) 27
Soluție Exercițiul 1 (Contorul) 27
Soluție Exercițiul 2 (Căutarea Live) 27
Soluție Exercițiul 3 (Ștergerea Paralelizată cu Promise.all) 28
Soluție Exercițiul 4 (Marcarea cu Checkbox și PATCH) 29
Soluție Exercițiul 5 (Sortare Alfabetică pe Front-End) 30
Soluție Exercițiul 6 (Sistem de Retry) 30

Capitolul 1: Construirea Scheletului (HTML5) - Fundația pentru Automatizare

🧠 Sesiunea 6 este **Workshop-ul Practic (Proiectul Capstone pentru Modulul 2)**. Aici luăm absolut tot ce am acumulat în Sesiunile 1 - 5 și asamblăm într-o singură aplicație web complet funcțională: **Task Tracker-ul final**. Această aplicație va deveni "subiectul" (SUT - Software Under Test) pe care îl vei testa automatizat începând cu Sesiunea 10!
Nu vom da copy-paste la un fișier uriaș. Vom construi structura aplicației pas cu pas, pentru a înțelege *de ce* fiecare linie de cod contează pentru Playwright.

🟤 Pasul 1: Boilerplate-ul și Header-ul (Navigația)

Scenariu Live: Orice aplicație are nevoie de un punct de pornire (declarația tipului de document) și de un antet (Header). În header vom pune titlul și un indicator de "Status Rețea" (care ne va fi util mai târziu pentru testele de latență).

🤔 ❓ **Cum testăm efectiv:**

- Deschide VS Code și creează un fișier nou numit **index.html**.
- Adaugă codul de mai jos (Structura de bază și Header-ul).
- Salvează și deschide fișierul în browser (Chrome).

- Pe ecran vei vedea doar textul brut "QA Task Tracker" și "API Online". Aplicația nu are viață încă.

Cod de scris în index.html:

```
<!DOCTYPE html>
<html lang="ro">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>QA Task Tracker Pro</title>
    <!-- Pregătim legătura pentru CSS-ul din Capitolul 2 -->
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- Antetul Aplicației -->
    <header class="navbar">
        <h1 id="logo">QA Task Tracker</h1>
        <div class="user-controls">
            <span id="network-status" class="status-indicator online">API
Online</span>
        </div>
    </header>

    <!-- Containerul principal unde vom pune restul aplicației -->
    <main class="container">
```

🟢 Pasul 2: Secțiunea de Input (Formularul) și Atributele de Testare

Scenariu Live: Acum adăugăm zona unde utilizatorul va introduce text. Acesta este cel mai important pas pentru un QA!

Observă atributele data-testid. Noi construim aplicația având *testabilitatea* în minte încă din secunda zero. Playwright nu va ghici niciodată unde să dea click, pentru că tocmai i-am desenat harta perfectă!

🤔 ❓ **Cum testăm efectiv:**

- Continuă să scrii codul de mai jos în index.html, fix sub <main class="container">.

- Salvează și dă refresh în browser.
- Observă butonul "Salvează". Chiar dacă arată urât momentan (fără CSS), încearcă să dai click pe el. Nu poți! Datorită atributului nativ HTML disabled, am blocat din start trimiterea de formulare goale.

Cod de continuat în index.html:

```
        <!-- Secțiunea de adăugare task -->
        <section class="card input-section">
            <h2>Adaugă un Scenariu Nou</h2>
            <form id="add-task-form">
                <div class="input-group">
                    <!-- Câmpul de text cu hook pentru Playwright -->
                    <input type="text" id="task-name" placeholder="ex:
Testare validare login..." data-testid="input-task-name">

                    <!-- Butonul pornește dezactivat pentru a forța
utilizatorul să scrie text -->
                    <button type="submit" id="submit-task-btn" data-
testid="submit-task-btn" disabled>Salvează</button>
                </div>

                <!-- Mesaj de eroare ascuns implicit (îl vom activa din JS)
-->
                <small id="error-msg" class="error-text" style="display:
none;" data-testid="error-message">Numele task-ului nu poate fi
gol!</small>
            </form>
        </section>
```

🟤 Pasul 3: Pregătirea Listei și Indicatorul de Latență (Spinner)

Scenariu Live: Robotul tău (Playwright) va interacționa cu rețeaua. Rețeaua are întârzieri. Trebuie să pregătim un element vizual (#loading-spinner) care să apară cât timp așteptăm răspunsul de la server, și o listă goală (<ul>) pe care o va popula JavaScript-ul ulterior.

🤔 ❓ Cum testăm efectiv:

- Adaugă acest ultim bloc de cod în index.html pentru a închide structura.
- Salvează și dă refresh în browser.

- Vei vedea textul "Se încarcă datele...". Aceasta este starea inițială, brută, a aplicației noastre. Acum că scheletul este gata, suntem pregătiți să trecem la design!

Cod final de adăugat în index.html:

```
        <!-- Secțiunea de listare task-uri -->
        <section class="card list-section">
            <h2>Scenarii de Testare (Live)</h2>

            <!-- Element vizual pentru Latența Asincronă -->
            <div id="loading-spinner" class="spinner" data-testid="loading-
spinner">Se încarcă datele de la server...</div>

            <!-- Lista goală: va fi populată de JavaScript în Capitolul 3 -
->
            <ul id="task-list" class="task-list" data-testid="task-list">
                <!-- Rândurile (<li>) vor fi injectate aici de JavaScript -
->
            </ul>
        </section>

    </main>

    <!-- Pregătim legătura pentru Logica din Capitolul 3 -->
    <script src="app.js"></script>
</body>
</html>
```

📖 Odată ce ai salvat fișierul și l-ai deschis în browser, vei vedea doar un text simplu, neformatat.

🧐 Aplicația nu are viață și nu are design încă.

Capitolul 2: Prezentarea Vizuală (CSS3) -

Construirea interfeței pas cu pas

> 🧠Acum că avem structura brută (HTML) din Capitolul 1, dacă te uiți în browser, aplicația arată ca un document de acum 20 de ani. Pentru ca testele noastre E2E să ruleze într-un mediu stabil, similar cu ceea ce vede un utilizator real în producție, trebuie să aplicăm stilizarea.
>
> Aici nu vom da copy-paste la un fișier uriaș. Vom construi design-ul pas cu pas direct în fișierul style.css, pentru a vedea live cum fiecare bloc de cod CSS ne rezolvă o problemă tehnică de interfață.

🟤 Pasul 1: Resetarea și Flexbox-ul pe Navbar

Scenariu Live: Marginile implicite ale browserului ne strică alinierea. Mai mult, vrem ca "Logo-ul" și "Statusul" din bara de navigație să fie împinse la extremele ecranului, pentru a nu se suprapune pe rezoluții mici.

🤔 ❓ **Cum testăm efectiv:**

- În VS Code, creează fișierul **style.css** în același folder cu index.html.
- Adaugă codul de mai jos și salvează.
- Dă refresh paginii în browser. Bara de sus va deveni albastru-închis, iar textul va fi perfect aliniat stânga-dreapta.
- **Test de QA (Responsiveness):** Micșorează fereastra browserului stânga-dreapta. Vei vedea că datorită lui justify-content: space-between, elementele nu se încalecă niciodată, păstrând o distanță sigură pentru robotul Playwright!

Cod de scris în style.css:

```
/* 1. Resetare de bază */
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    font-family: 'Segoe UI', Tahoma, sans-serif;
}

body {
    background-color: #f0f2f5;
    color: #333;
```

```
}

/* 2. Bara de Navigație (Flexbox) */
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background-color: #1a252f;
    color: white;
    padding: 15px 30px;
}

.status-indicator {
    font-size: 12px;
    padding: 5px 10px;
    border-radius: 20px;
    font-weight: bold;
}
.online { background-color: #27ae60; color: white; }
```

🟢 Pasul 2: Formularul și Stările Butonului (:disabled vs :hover)

Scenariu Live: Secțiunea de input arată dezordonat. Mai important pentru noi ca testeri: butonul de "Salvează" este dezactivat în HTML (disabled), dar vizual arată exact ca un buton normal. Trebuie să îi schimbăm starea vizuală clar!

🤔 ❓ Cum testăm efectiv:

- Adaugă codul de mai jos continuând în style.css și dă refresh în browser.
- Privește butonul "Salvează". Acum este gri și cursorul se transformă într-un semn de "interzis" (not-allowed) când treci cu mouse-ul peste el. Aceasta este o indicație vizuală clară că starea HTML disabled funcționează.
- Mesajul de eroare ("Numele task-ului nu poate fi gol!") a primit culoarea roșie specifică alertelor.

Cod de adăugat în style.css:

```
/* 3. Containerul Principal și Cardurile */
.container { max-width: 800px; margin: 40px auto; padding: 0 20px; }

.card {
    background: white;
    padding: 25px;
```

```
    border-radius: 10px;
    box-shadow: 0 4px 10px rgba(0,0,0,0.05);
    margin-bottom: 30px;
}
.card h2 { margin-bottom: 20px; font-size: 20px; border-bottom: 2px solid
#ecf0f1; padding-bottom: 10px; }

/* 4. Formularul de input */
.input-group { display: flex; gap: 15px; }

input[type="text"] {
    flex: 1;
    padding: 12px;
    border: 2px solid #dfe6e9;
    border-radius: 6px;
    font-size: 16px;
}

button {
    background-color: #3498db;
    color: white;
    border: none;
    padding: 12px 25px;
    border-radius: 6px;
    cursor: pointer;
    font-weight: bold;
}

/* Starea Normală de Hover */
button:hover:not(:disabled) { background-color: #2980b9; }

/* Starea "Disabled" - crucială pentru testele UI */
button:disabled {
    background-color: #bdc3c7;
    cursor: not-allowed;
}

.error-text { color: #e74c3c; margin-top: 8px; font-size: 13px; font-
weight: bold; }
```

🟢 Pasul 3: Pregătirea Lista de Task-uri (Latența & Lista)

Scenariu Live: Elementul vizual de "Latență" (Spinner-ul) trebuie să iasă în evidență pentru a simula așteptarea pe rețea. De asemenea, trebuie să pregătim CSS-ul pentru viitoarele rânduri

 pe care le va genera JavaScript-ul, asigurându-ne că butonul de "Ștergere" este mereu lipit de marginea din dreapta.

🤔 ❓ **Cum testăm efectiv:**

- Adaugă codul final în style.css.
- Dă refresh. Mesajul "Se încarcă datele..." apare acum ca un banner portocaliu distinct.
- *Trick de testare manuală:* Pentru a vedea cum va arăta un task generat de JS, du-te o secundă în index.html și adaugă manual între etichetele <ul id="task-list"> ... următoarea linie:
  <li class="task-item">Test Manual<button class="btn-delete">Șterge</button>
- Dă refresh. Vei vedea că datorită lui justify-content: space-between pe clasa .task-item, butonul roșu de ștergere stă perfect aliniat la dreapta! (Nu uita să ștergi linia manuală din HTML după ce te-ai convins că funcționează).

Cod de adăugat în style.css:

```
/* 5. Feedback Vizual pentru acțiuni asincrone (Spinner) */
.spinner {
    color: #f39c12;
    text-align: center;
    margin: 20px 0;
    padding: 10px;
    background-color: #fcf3cf;
    border-radius: 6px;
}

/* 6. Lista de Task-uri (Pregătire pentru JS) */
.task-list { list-style: none; }

.task-item {
    display: flex;
    justify-content: space-between; /* Împinge textul la stânga și butonul
la dreapta */
    align-items: center;
    padding: 15px 20px;
    background-color: #f8f9fa;
    border: 1px solid #e9ecef;
    border-radius: 6px;
    margin-bottom: 12px;
}

.task-title { font-size: 15px; font-weight: 500; }
```

```
.btn-delete { background-color: #e74c3c; font-size: 13px; padding: 8px
15px; }
.btn-delete:hover { background-color: #c0392b; }
```

📈 Aplicația noastră are acum "față" de Enterprise! Următorul pas este să îi montăm "creierul" în Capitolul 3.

Capitolul 3: Logica Aplicației (JavaScript) - "Creierul" Asincron

> 🧠 Avem structura (HTML) și avem design-ul (CSS). Dar aplicația noastră este ca o mașină frumoasă fără motor. Acum vom scrie fișierul app.js pas cu pas, pentru a conecta interfața la un server real (API) și a face butoanele să funcționeze.
>
> Acesta este momentul în care totul se leagă! Codul pe care îl scriem aici dictează comportamentul pe care tu, ca viitor Automation QA, îl vei testa cu Playwright.

🟤 Pasul 1: Selectoarele DOM și Funcția de Randare (Reutilizabilitate)

Scenariu Live: Primul lucru pe care trebuie să-l facă JS-ul este să facă "cunoștință" cu elementele din HTML. Apoi, vom crea o funcție (o rețetă) care știe cum să deseneze un rând HTML (<li>) pentru un task. O vom scrie o singură dată și o vom refolosi și la adăugare, și la încărcarea inițială!

🤔 ❓ Cum testăm efectiv:

- În VS Code, în același folder, creează un fișier nou numit **app.js**.
- Adaugă codul de mai jos și salvează.
- Dă refresh paginii în browser. Nu se va schimba nimic vizual încă, dar dacă deschizi Consola (F12) și nu ai nicio eroare cu roșu, înseamnă că JS-ul a "agățat" corect elementele din HTML!

Cod de scris în app.js:

```
// 1. SELECTOARE (Prindem elementele din HTML)
const form = document.querySelector('#add-task-form');
const inputNode = document.querySelector('#task-name');
const submitBtn = document.querySelector('#submit-task-btn');
const errorMsg = document.querySelector('#error-msg');
const taskList = document.querySelector('#task-list');
const spinner = document.querySelector('#loading-spinner');

// API-ul nostru public pentru testare
const API_URL = 'https://jsonplaceholder.typicode.com/todos';

// 2. FUNCȚIE REUTILIZABILĂ: Desenarea unui rând HTML în pagină
const deseneazaTaskInDOM = (id, titlu) => {
    // Creăm un element <li> în memorie
    const li = document.createElement('li');
    li.className = 'task-item';

    // Injectăm HTML-ul intern (inclusiv atributele de testare pentru
Playwright!)
    li.innerHTML = `
        <span class="task-title">titlu</span><buttonclass="btn-delete"data-
testid="delete-btn-{id}">Șterge</button>
    `;

    // Îl lipim fizic în lista <ul> de pe ecran
    taskList.appendChild(li);
};
```

🟤 Pasul 2: Încărcarea datelor la pornire (GET)

Scenariu Live: Când deschizi aplicația, ea nu trebuie să fie goală. Trebuie să aducă automat ultimele task-uri salvate în baza de date și, foarte important, să ascundă banner-ul portocaliu cu "Se încarcă..." odată ce datele au sosit.

🤔 ❓ Cum testăm efectiv:

- Continuă să adaugi codul de mai jos în app.js.
- Salvează și dă refresh paginii în browser.

- **Urmărește cu atenție ecranul:** Vei vedea banner-ul portocaliu ("Se încarcă datele...") pentru o fracțiune de secundă. Apoi, acesta dispare și în locul lui apar 5 task-uri aduse *live* de pe internet!

Cod de continuat în app.js:

```
// 3. GET: Aducerea datelor inițiale de la Server
const incarcaDateServer = async () => {
    try {
        // Cerem primele 5 task-uri de la API (Așteptăm "Chelnerul")
        const response = await fetch(`${API_URL}?_limit=5`);

        if (!response.ok) throw new Error("A picat serverul!");

        // Despachetăm datele JSON
        const data = await response.json();

        // Odată ce avem datele, ascundem spinner-ul
        spinner.style.display = 'none';

        // Parcurgem lista și desenăm fiecare task
        data.forEach(task => {
            deseneazaTaskInDOM(task.id, task.title);
        });

    } catch (error) {
        spinner.textContent = "Eroare la aducerea datelor!";
        spinner.style.backgroundColor = "#f8d7da";
        spinner.style.color = "#721c24";
        console.error(error);
    }
};

// Pornim funcția imediat ce scriptul se execută
incarcaDateServer();
```

🟤 Pasul 3: Salvarea unui Task Nou (POST & Latență)

Scenariu Live: Când omul completează formularul și apasă "Salvează", trebuie să interceptăm trimiterea, să trimitem pachetul la server prin metoda POST, să arătăm un indicator vizual pe buton ("Se salvează...") și abia la final să desenăm noul task în listă.

🤓 ❓ **Cum testăm efectiv:**

- Adaugă acest cod în app.js.
- Dă refresh în browser.
- Pentru că butonul "Salvează" este dezactivat implicit (din HTML), dă click pe cuvântul disabled din fișierul tău index.html și șterge-l temporar doar ca să putem testa acest pas.
- Scrie ceva în căsuță și dă click pe "Salvează".
- Observă cum butonul își schimbă textul, iar după o scurtă latență de rețea, task-ul tău apare ultimul în listă, iar căsuța se golește automat!

Cod de continuat în app.js:

```
// 4. POST: Adăugarea unui task nou pe rețea
form.addEventListener('submit', async (e) => {
    e.preventDefault(); // Foarte important: Oprește refresh-ul paginii!

    const taskTitle = inputNode.value.trim();

    // UX Feedback: Blocăm butonul și îi schimbăm textul cât timp așteptăm
rețeaua
    submitBtn.textContent = "Se salvează...";
    submitBtn.disabled = true;

    try {
        const response = await fetch(API_URL, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                title: taskTitle,
                completed: false,
                userId: 1
            })
        });

        if (response.status === 201) {
            const newTask = await response.json();
```

```
            // Serverul ne-a dat înapoi ID-ul nou generat (ex: 201).
Desenăm task-ul!
            deseneazaTaskInDOM(newTask.id, newTask.title);

            // Curățăm formularul
            inputNode.value = '';
        }
    } catch (error) {
        alert("Eroare la salvarea pe rețea!");
    } finally {
        // Restaurăm butonul la starea inițială, indiferent dacă a fost
succes sau eroare
        submitBtn.textContent = "Salvează";
        submitBtn.disabled = true; // Îl lăsăm dezactivat pentru că input-
ul e acum gol
    }
});
```

🟢 Pasul 4: Validarea Formularului (Evenimentul de 'input')

Scenariu Live: Nu vrem ca utilizatorii să poată trimite task-uri goale. Trebuie să "ascultăm" la fiecare literă pe care o tastează. Dacă șterge tot, blocăm butonul de salvare și afișăm mesajul de eroare roșu.

🤔 ❓ Cum testăm efectiv:

- Adaugă acest ultim bloc de cod în app.js (și dacă ai șters disabled din index.html la pasul anterior, pune-l la loc!).
- Salvează și dă refresh.
- Dă click pe input, tastează cuvântul "Test". Butonul "Salvează" se activează imediat (se face albastru).
- Folosește Backspace și șterge cuvântul "Test". Instantaneu, butonul redevine gri (disabled), iar sub el apare mesajul roșu de eroare.
- Aceasta este o funcționalitate Front-end 100% matură pe care tu, ca QA, o vei testa automatizat!

Cod final de adăugat în app.js:

```
// 5. EVENIMENTE: Validare "Live" la fiecare apăsare de tastă
inputNode.addEventListener('input', () => {
    // Verificăm dacă există măcar o literă scrisă
    if (inputNode.value.trim().length > 0) {
        submitBtn.disabled = false;
        errorMsg.style.display = 'none';
    } else {
        // Dacă e gol, blocăm butonul și arătăm eroarea
        submitBtn.disabled = true;
        errorMsg.style.display = 'block';
    }
});
```

🎉 **FELICITĂRI MAJORE!** Ai terminat de construit aplicația Task Tracker E2E! Acum ai o platformă reală, stabilă, cu latență de rețea, validări și o structură impecabilă de atribute data-testid. Aplicația ta este pregătită. La Modulul 4, tu și robotul Playwright veți lansa primele atacuri automatizate asupra ei!

Capitolul 4: Ștergerea pe Rețea (DELETE) și Checklist-ul Final de QA

> 🧠 Aplicația noastră arată și funcționează excelent pe partea de adăugare și citire (GET & POST). Dar, pentru un scenariu CRUD complet, ne lipsește ultima și cea mai destructivă acțiune: **Ștergerea (DELETE)**.
>
> Mai mult, după ce finalizăm codul, vom schimba complet "pălăria" de Developer cu cea de Inginer QA și vom rula un Checklist Manual înainte de a declara aplicația pregătită pentru automatizarea cu Playwright.

🟤 Pasul 1: Ștergerea reală (Metoda HTTP DELETE & Event Delegation)

Scenariu Live: Butoanele de "Șterge" sunt create dinamic de JavaScript (ele nu există în HTML-ul inițial, ci sunt desenate abia după ce vin datele de la server). Dacă încercăm să punem un addEventListener clasic pe ele la început, va da eroare.

Soluția este **Event Delegation**: punem ascultătorul pe întreaga listă <ul> și verificăm dacă click-ul a provenit de la un buton cu clasa btn-delete. Apoi, vom extrage ID-ul task-ului și vom trimite o cerere HTTP DELETE către server.

🤔 ❓ **Cum testăm efectiv:**

- Adaugă acest ultim bloc de cod la finalul fișierului tău app.js.
- Salvează și dă refresh în browser.
- Dă click pe butonul roșu "Șterge" al oricărui task. Observă cum butonul se dezactivează, textul i se schimbă în "Se șterge...", și abia după ce serverul răspunde cu succes, întregul rând dispare de pe ecran.

Cod final de adăugat în app.js:

```
// 6. DELETE: Ștergerea unui task pe rețea și din DOM
taskList.addEventListener('click', async (e) => {
    // Verificăm dacă elementul pe care am dat click are clasa 'btn-delete'
    if (e.target.classList.contains('btn-delete')) {

        const deleteBtn = e.target;
        const randTask = deleteBtn.closest('.task-item');

        // Extragem ID-ul task-ului din atributul de testare (ex: "delete-
btn-1" -> "1")
        const testId = deleteBtn.getAttribute('data-testid');
        const taskId = testId.split('-')[2];

        // UX (Feedback vizual): Blocăm butonul și arătăm că se procesează
        deleteBtn.textContent = 'Se șterge...';
        deleteBtn.disabled = true;
        randTask.style.opacity = '0.6'; // Facem rândul "fantomă"

        try {
            // Trimitem cererea de ștergere către Server
            const response = await fetch(`
APIURL/
{taskId}`, {
                method: 'DELETE'
            });

            if (response.ok) {
                // Dacă serverul a confirmat ștergerea (HTTP 200/204),
distrugem fizic elementul din DOM
                randTask.remove();
            } else {
                throw new Error("Serverul a refuzat ștergerea!");
            }
        } catch (error) {
            // Dacă rețeaua pică, restaurăm elementul la starea inițială!
            alert("Eroare de rețea la ștergere!");
            deleteBtn.textContent = 'Șterge';
            deleteBtn.disabled = false;
            randTask.style.opacity = '1';
        }
    }
});
```

🟤 Pasul 2: Manual QA Checklist (SUT Sign-off)

Acum că ai construit complet aplicația, tu ești QA-ul! Înainte de a scrie cod automatizat în Playwright (ceea ce vom face în Modulul 4), trebuie să te asiguri că SUT-ul tău (Software Under Test) este testabil și reacționează corect la scenariile umane.

Execută manual aceste 4 teste fundamentale în browserul tău:

☑ **Testul 1: Încărcarea Inițială (Starea de Interceptare GET)**

- *Acțiune:* Deschide/dă Refresh la pagină.
- *Rezultat așteptat:* Mesajul portocaliu "Se încarcă datele..." apare. Ulterior dispare și apar 5 rânduri de task-uri aduse live de pe API. *(La automatizare, aici vom învăța să testăm starea elementelor cu await expect(locator).toBeVisible())*.

☑ **Testul 2: Validarea Câmpului (Boundary & State Testing)**

- *Acțiune:* Dă click în câmpul de text, scrie un cuvânt, apoi șterge-l folosind Backspace până când câmpul este complet gol.
- *Rezultat așteptat:* Imediat ce câmpul e gol, butonul de "Salvează" devine gri (starea disabled), iar sub el apare mesajul de eroare roșu. *(Aceasta este validare de UI pură)*.

☑ **Testul 3: Adăugarea (POST & Așteptarea Activă)**

- *Acțiune:* Scrie "Testare E2E Playwright" și apasă Salvează.
- *Rezultat așteptat:* Butonul se schimbă în "Se salvează...". După latența de rețea, noul rând apare la finalul listei, iar câmpul de text se golește automat. *(În Playwright, vom scrie un asert care așteaptă ca lista să își crească lungimea cu +1 element)*.

☑ **Testul 4: Ștergerea cu Tratarea Latenței (DELETE)**

- *Acțiune:* Dă click pe butonul roșu "Șterge" de lângă un task.
- *Rezultat așteptat:* Elementul se estompează vizual, butonul scrie "Se șterge...", și doar după ce se rezolvă promisiunea HTTP, elementul dispare complet. *(Acesta este cel mai greu test în automatizare: verificarea dispariției unui element din DOM folosind not.toBeAttached())*.

🎉 **FINAL DE MODUL 2!**

Acum știi exact cu ce se mănâncă web-ul. Uită-te atent la atributele data-testid pe care le-ai adăugat în HTML și JS. Ele sunt motivul pentru care vei iubi această aplicație în modulele

următoare. Playwright nu va ghici niciodată unde să dea click; noi tocmai i-am desenat harta arhitecturală perfectă!

Aplicația ta este oficial gata! Urmează Modulul 3, unde vom face upgrade-ul absolut: introducerea **TypeScript-ului**.

Capitolul 5 (Bonus): Modul "Chaos" - Inamicul

testelor automate (Flaky Pop-up)

📝 Aplicația noastră funcționează perfect. Dar în lumea reală (pe producție), aplicațiile nu sunt mereu previzibile. Apar bannere de Cookies, promoții de Black Friday sau ferestre de tip "Vrei să te abonezi la newsletter?" care se încarcă prin scripturi externe, cu o întârziere aleatoare.

Dacă scrii un test automatizat care dă click pe "Salvează", iar exact în acea milisecundă apare un pop-up peste butonul tău, testul va pica cu faimoasa eroare: Element is intercepted by another element.

Hai să programăm intenționat acest "coșmar" în aplicația noastră, pentru a ne antrena mușchii de QA Automation în modulele următoare!

🔘 Pasul 1: Stilizarea Pop-up-ului (Z-Index și Overlays)

Scenariu Live: Fereastra pop-up trebuie să plutească deasupra întregii aplicații și să aibă un fundal semi-transparent care să blocheze temporar interacțiunea cu restul paginii (un "Overlay").

🤔 ❓ Cum testăm efectiv:

- Deschide fișierul tău **style.css** și adaugă codul de mai jos chiar la final.
- Salvează fișierul. Vizual nu se va întâmpla nimic încă, deoarece elementul HTML nu există. Pregătim doar terenul pentru JavaScript.

Cod final de adăugat în style.css:

```
/* 7. Modul Chaos: Flaky Promo Pop-up */
.overlay-modal {
    position: fixed;
    top: 0; left: 0; right: 0; bottom: 0;
    background-color: rgba(0, 0, 0, 0.7); /* Fundal întunecat transparent
*/
    z-index: 9999; /* Ne asigurăm că e deasupra tuturor elementelor */
    display: flex;
    justify-content: center;
```

```
    align-items: center;
    backdrop-filter: blur(3px);
}

.promo-box {
    background: white;
    padding: 30px;
    border-radius: 10px;
    text-align: center;
    box-shadow: 0 10px 25px rgba(0,0,0,0.5);
    max-width: 400px;
    animation: popIn 0.3s ease-out forwards; /* O mică animație vizuală */
}

.promo-box h3 { color: #e74c3c; margin-bottom: 15px; font-size: 22px; }
.promo-box p { margin-bottom: 20px; font-size: 15px; }

.btn-close-promo {
    background-color: #2c3e50;
    color: white;
    width: 100%;
}
.btn-close-promo:hover:not(:disabled) { background-color: #1a252f; }

@keyframes popIn {
    0% { transform: scale(0.5); opacity: 0; }
    100% { transform: scale(1); opacity: 1; }
}
```

🟤 Pasul 2: JavaScript Asincron Aleatoriu (Injecția)

Scenariu Live: Vom folosi metoda setTimeout combinată cu funcția Math.random(). Vom spune aplicației: "După un timp aleatoriu între 2 și 6 secunde de la deschiderea paginii, generează brusc fereastra de Promoții. Adaugă și un buton de închidere care distruge fereastra."

🤔 ❓ Cum testăm efectiv:

- Deschide fișierul app.js și lipește acest bloc de cod exact la finalul fișierului.
- Salvează și dă **Refresh** în browser.
- Încearcă să adaugi un task nou. Scrie rapid, apasă "Salvează". Dă un nou refresh și încearcă din nou.

- La un moment dat (între 2 și 6 secunde), exact când te aștepți mai puțin, ecranul se va întuneca și va apărea mesajul Promoțional, blocându-ți acțiunile. Apasă pe "Nu, mulțumesc" pentru a-l închide și a-ți continua treaba.

Cod final de adăugat în app.js:

```
// 7. BONUS: Modul Chaos - Pop-up aleatoriu care interceptează testele UI!
const initChaosMode = () => {
    // Calculăm un timp aleatoriu de așteptare între 2000ms și 6000ms
    const randomDelay = Math.floor(Math.random() * 4000) + 2000;

    setTimeout(() => {
        // Creăm elementul container (Overlay-ul)
        const modal = document.createElement('div');
        modal.className = 'overlay-modal';
        modal.setAttribute('data-testid', 'promo-overlay');

        // Injectăm cutia promoțională și butonul de închidere
        modal.innerHTML = `
            <div class="promo-box">
                <h3>Ofertă Specială! 🚀</h3>
                <p>Fă upgrade la versiunea Premium pentru a adăuga task-uri
nelimitate!</p>
                <button class="btn-close-promo" data-testid="close-promo-
btn">Nu, mulțumesc</button>
            </div>
        `;

        // Atașăm evenimentul de închidere
        const closeBtn = modal.querySelector('.btn-close-promo');
        closeBtn.addEventListener('click', () => {
            modal.remove(); // Distrugem modalul când utilizatorul dă click
        });

        // "Aruncăm" modalul fizic în pagină (la finalul tag-ului <body>)
        document.body.appendChild(modal);

        console.warn(`[Chaos Mode] Promo modal a apărut după
${randomDelay}ms!`);
    }, randomDelay);
};
```

```
// Pornim modul Chaos!
initChaosMode();
```

🟤 Concluzia QA-ului (SUT Sign-off)

> 📈 Ce ai făcut acum nu este o simplă joacă. Este un **Edge Case (Caz limită)** clasic din ingineria Software.
>
> Când vom scrie testele automate în Playwright, robotul tău va trebui să fie suficient de inteligent încât să ia o decizie:
>
> - *"Vreau să apăs pe Salvare. Există un modal pe ecran care mă blochează? Dacă da, îl închid întâi, și abia apoi dau Salvare."*
>
> Playwright are o funcție specială pentru asta (numită page.addLocatorHandler()), care acționează ca un "câine de pază" asincron ce vânează aceste ferestre instabile pe parcursul rulării testului. Noi tocmai i-am construit terenul perfect de antrenament!

Workbook S6 - Extinderea Aplicației "Task Tracker"

📈 **Scopul acestui material:** Acum că ai o aplicație E2E funcțională (Task Tracker), o vom extinde la nivel de producție! În automatizare, aplicațiile primesc constant noi "Features" (funcționalități). Rolul tău aici este să scrii codul pentru aceste funcționalități noi, pregătind astfel terenul pentru scenarii de testare complexe în Playwright.

Toate exercițiile de mai jos se vor construi pe fișierele index.html și app.js pe care le-ai finalizat în Capitolul 4 al workshop-ului live.

💻 Exercițiul 1: Contorul de Task-uri (State Management)

Scenariul de Business: Utilizatorii vor să vadă rapid câte task-uri au în listă, fără să le numere manual.

Relevanța QA: În Playwright, vei scrie adesea asertări de tipul: expect(locator).toHaveText("3 Task-uri").

🤔 ❓ **Misiunea ta:**

- Deschide index.html și adaugă un 0 Task-uri chiar lângă titlul "Scenarii de Testare (Live)".
- Deschide app.js. Creează o funcție actualizeazaContor = () => { ... }.
- Această funcție trebuie să numere câte elemente <li> (cu clasa .task-item) există în acel moment pe ecran folosind document.querySelectorAll().
- Actualizează textul din #task-counter cu noul număr.
- **CRITIC:** Apelează această funcție în 3 locuri cheie din codul tău existent: după ce se încarcă datele inițiale (GET), după ce adaugi un task nou (POST) și după ce ștergi un task (DELETE).

💻 Exercițiul 2: Bara de Căutare (Filtrare live în DOM)

Scenariul de Business: Dacă lista are 50 de scenarii, e greu să găsești unul. Avem nevoie de un câmp de căutare "Live" (Search).

Relevanța QA: Testarea filtrelor de search este unul dintre cele mai comune scenarii E2E.

Trebuie să validezi că atunci când scrii "Login", elementele care nu conțin acest cuvânt dispar de pe ecran.

🤯 ❓ **Misiunea ta:**

- În index.html, adaugă un <input type="text" id="search-box" placeholder="Caută task..."> deasupra listei de task-uri.
- În app.js, adaugă un ascultător de evenimente (input) pe acest #search-box.
- De fiecare dată când utilizatorul tastează o literă:
  - Ia valoarea scrisă și transform-o în litere mici (.toLowerCase()).
  - Parcurge absolut toate -urile din pagină (folosind o buclă sau forEach).
  - Dacă textul din interiorul task-ului *include* textul căutat, lasă elementul vizibil (display: flex).
  - Dacă nu, ascunde-l (display: none).

💻 Exercițiul 3: "Șterge Tot" prin Paralelizare (Senior Level)

Scenariul de Business: Vrem un buton roșu periculos: "Clear All", care șterge toate task-urile dintr-un singur click.

Relevanța QA: Cum testezi că 5 elemente au fost șterse de pe rețea simultan? Folosind API-ul și paralelizarea!

🤯 ❓ **Misiunea ta:**

- În index.html, adaugă un buton <button id="clear-all-btn">Șterge Tot</button>.
- În app.js, pune un eveniment de click pe el.
- Când se dă click, extrage TOATE ID-urile task-urilor prezente pe ecran într-un Array.
- Folosește o buclă pentru a crea o cerere fetch cu metoda DELETE pentru fiecare ID, dar **NU** le da await pe rând. Salvează promisiunile (cererile) într-un array.
- Folosește await Promise.all(arrayCereri) pentru a lansa toate cele 5-10 cereri de ștergere pe rețea *simultan*!
- După ce a trecut Promise.all, golește lista HTML vizuală (taskList.innerHTML = '';).

💻 Exercițiul 4: Finalizarea unui Task (Metoda

PATCH)

Scenariul de Business: Utilizatorii trebuie să poată bifa un task ca fiind "Rezolvat". Textul trebuie să fie tăiat cu o linie (strikethrough), iar serverul trebuie notificat.

Relevanța QA: Aici vei testa un Update (modificarea stării unui element existent). Robotul tău va bifa căsuța și va verifica dacă elementului i s-a aplicat clasa corectă de CSS.

🤔 ❓ **Misiunea ta:**

- În funcția ta deseneazaTaskInDOM, adaugă un <input type="checkbox" class="check-task"> la începutul fiecărui rând HTML.
- În zona unde folosești Event Delegation pentru lista de task-uri (unde asculți click-ul pentru Ștergere), ascultă și pentru evenimentul de change (schimbare) pe acel checkbox.
- Dacă checkbox-ul a fost bifat, taie textul task-ului (textDecoration = 'line-through'). Dacă a fost debifat, revino la normal (textDecoration = 'none').
- **Extra (Rețea):** Trimite o cerere fetch cu metoda PATCH către API (ex: https://jsonplaceholder.typicode.com/todos/ID), trimițând în Body { completed: true }.

💻 Exercițiul 5: Sortare Alfabetică în DOM

Scenariul de Business: Utilizatorii au o listă dezordonată și vor un buton magic care să ordoneze task-urile de la A la Z.

Relevanța QA: Validarea sortării este o provocare celebră la interviuri. Robotul tău va extrage toate textele într-un array, le va sorta el în memorie și va compara cu ordinea afișată pe ecran.

🤔 ❓ **Misiunea ta:**

- Adaugă un <button id="sort-btn">Sortează A-Z</button> în HTML.
- În JS, extrage toate elementele <li> într-un Array real (folosind Array.from()).
- Folosește metoda .sort() din JavaScript pentru a ordona elementele HTML în funcție de textul din ele (localeCompare).
- Golește -ul existent.
- Fă un forEach pe array-ul tău sortat și lipește rândurile înapoi în DOM! Ele vor apărea ordonate!

💻 Exercițiul 6: Sistem de "Retry" (Auto-

Vindecare)

Scenariul de Business: Dacă prima cerere de aducere a datelor (GET) pică din cauza internetului, nu vrem doar să afișăm o eroare, ci să oferim un buton de "Încearcă din nou".

Relevanța QA: Vei testa cum reacționează aplicația la erori de server 500 simulate.

🤔 ❓ **Misiunea ta:**

- Când cererea ta GET (de la încărcarea paginii) prinde o eroare în blocul catch, desenează în HTML un buton <button id="retry-btn">Reîncearcă</button>.
- Pune un addEventListener pe acest buton care, la apăsare, curăță eroarea și apelează funcția incarcaDateServer() din nou!
- (Test): Strică intenționat URL-ul ca să pice, vezi butonul de Retry, repară URL-ul, dă click pe Retry și vezi cum datele vin cu succes!

🔑 Cheile de Rezolvare (Soluții)

Nu citi mai departe până nu ai încercat să rezolvi singur exercițiile! Ai voie să cauți pe Google metode JS specifice!

Soluție Exercițiul 1 (Contorul)

```
// Adaugă asta la începutul app.js
const counterElement = document.querySelector('#task-counter');

const actualizeazaContor = () => {
    const numarTaskuri = document.querySelectorAll('.task-item').length;
    if(counterElement) counterElement.textContent = `${numarTaskuri} Task-
uri`;
};

// Exemplu de apelare după finalizarea încărcării inițiale (în funcția
incarcaDateServer):
// ...
data.forEach(task => deseneazaTaskInDOM(task.id, task.title));
actualizeazaContor(); // <--- Apelează aici! (Și în POST, și în DELETE)
```

Soluție Exercițiul 2 (Căutarea Live)

```
const searchBox = document.querySelector('#search-box');

searchBox.addEventListener('input', (e) => {
    const textCautat = e.target.value.toLowerCase();
    const toateTaskurile = document.querySelectorAll('.task-item');

    toateTaskurile.forEach(task => {
        const titluTask = task.querySelector('.task-
title').textContent.toLowerCase();

        if (titluTask.includes(textCautat)) {
            task.style.display = 'flex'; // Arată elementul
        } else {
            task.style.display = 'none'; // Ascunde elementul
        }
    });
});
```

Soluție Exercițiul 3 (Ștergerea Paralelizată cu Promise.all)

```
const clearAllBtn = document.querySelector('#clear-all-btn');

clearAllBtn.addEventListener('click', async () => {
    const toateTaskurile = document.querySelectorAll('.task-item');
    if (toateTaskurile.length === 0) return;

    clearAllBtn.textContent = "Se șterge tot...";
    clearAllBtn.disabled = true;

    try {
        const cereriDelete = [];

        // Adunăm toate "brichetele" (cererile) fără să le aprindem încă
        toateTaskurile.forEach(task => {
            const testId = task.querySelector('.btn-
delete').getAttribute('data-testid');
            const taskId = testId.split('-')[2];
            const cerereRețea = fetch(`APIURL/{taskId}`, { method: 'DELETE'
});
            cereriDelete.push(cerereRețea);
        });

        // Aprindem toate brichetele simultan! (Paralelizare)
```

```
        await Promise.all(cereriDelete);

        // Dacă rețeaua a fost ok, curățăm și DOM-ul
        taskList.innerHTML = '';
        actualizeazaContor();
    } catch (eroare) {
        alert("O eroare a apărut la ștergerea masivă!");
    } finally {
        clearAllBtn.textContent = "Șterge Tot";
        clearAllBtn.disabled = false;
    }
});
```

Soluție Exercițiul 4 (Marcarea cu Checkbox și PATCH)

```
// Adăugăm delegare pentru evenimentul 'change' pe lista principală (pentru
checkbox-uri)
taskList.addEventListener('change', async (e) => {
    if (e.target.classList.contains('check-task')) {
        const randApasat = e.target.closest('.task-item');
        const titluElement = randApasat.querySelector('.task-title');

        // Extragem ID-ul la fel cum o făceam pentru ștergere
        const deleteBtn = randApasat.querySelector('.btn-delete');
        const taskId = deleteBtn.getAttribute('data-testid').split('-')[2];

        const esteBifat = e.target.checked;

        // Optimistic UI Update (Schimbăm vizualul instant, ca să nu
așteapte omul după net)
        titluElement.style.textDecoration = esteBifat ? 'line-through' :
'none';
        titluElement.style.color = esteBifat ? '#7f8c8d' : '#333';

        try {
            // Trimitem update-ul la server prin PATCH (actualizare
parțială)
            await fetch(`APIURL/{taskId}`, {
                method: 'PATCH',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ completed: esteBifat })
            });
        } catch (error) {
```

```
            console.error("Eroare la actualizarea statusului pe rețea",
error);
            // Dacă pică netul, dăm revert vizual
            e.target.checked = !esteBifat;
            titluElement.style.textDecoration = !esteBifat ? 'line-through'
: 'none';
        }
    }
});
```

Soluție Exercițiul 5 (Sortare Alfabetică pe Front-End)

```
const sortBtn = document.querySelector('#sort-btn');

sortBtn.addEventListener('click', () => {
    // 1. Luăm NodeList-ul și îl convertim în Array real ca să putem folosi
.sort()
    const arrayTaskuri = Array.from(document.querySelectorAll('.task-
item'));

    // 2. Sortăm array-ul în memorie, comparând textul din fiecare <li>
    arrayTaskuri.sort((a, b) => {
        const textA = a.querySelector('.task-
title').textContent.toLowerCase();
        const textB = b.querySelector('.task-
title').textContent.toLowerCase();
        return textA.localeCompare(textB); // Metodă built-in pentru a
sorta alfabetic stringuri
    });

    // 3. Golim ecranul HTML
    taskList.innerHTML = '';

    // 4. Punem elementele înapoi în DOM, acum ordonate!
    arrayTaskuri.forEach(task => taskList.appendChild(task));
});
```

Soluție Exercițiul 6 (Sistem de Retry)

```
// În interiorul funcției tale de fetch inițial (ex: incarcaDateServer), în
zona catch:
// ...
    } catch (error) {
```

```
        spinner.textContent = "Eroare la aducerea datelor! ";
        spinner.style.backgroundColor = "#f8d7da";

        // Creăm dinamic butonul de Retry
        const retryBtn = document.createElement('button');
        retryBtn.textContent = "Reîncearcă";
        retryBtn.className = "btn-retry"; // Asigură-te că ai un CSS pentru
el

        // Când se apasă, golim eroarea și reluăm procesul
        retryBtn.addEventListener('click', () => {
            spinner.textContent = "Se încarcă datele de la server...";
            spinner.style.backgroundColor = "#fcf3cf";
            incarcaDateServer(); // Apelăm funcția recursiv!
        });

        spinner.appendChild(retryBtn);
    }
// ...
```